Machine Learning–Based Software Effort Estimation

Rahaf Atari, Abeer Hussein, Aysha Mohammad, Ahmad Hussein
Electrical And Computer Engineering, Birzeit University, Ramallah, Palestine
Email: 1210326@student.birzeit.edu

***Abstract*—Software effort estimation is a critical input to project budgeting, scheduling, and resource planning. Inaccurate estimates can lead to cost overruns, delivery delays, and inefficient staffing decisions. This paper investigates machine learning (ML) regression models for predicting software development effort using a historical project dataset (81 instances). We compare multiple ML models against a baseline estimator and evaluate performance using standard regression metrics (MAE, RMSE, R^2) and effort-estimation metrics (MMRE, PRED(25)). To avoid optimistic bias during tuning, we use nested cross-validation and report the resulting performance estimates. Finally, we translate forecasting outcomes into practical project management decisions such as staffing adjustments, schedule impact, and contingency reserve recommendations.**

***Index Terms*—software effort estimation, project management, machine learning, cost estimation, scheduling, risk management**

I. Introduction

Effort estimation influences project cost, schedule, staffing, and risk reserves. Underestimation increases schedule pressure and delivery risk, while overestimation wastes budget and resources. Traditional approaches (expert judgment, analogy-based estimation, and parametric models) are useful but often struggle to generalize across organizations and project types [1], [2]. ML models trained on historical data can learn nonlinear relationships between project attributes and effort, potentially improving accuracy and supporting evidence-based planning [3], [4].

This paper is written from a project management perspective: effort forecasts matter only if they improve planning decisions. Therefore, beyond predictive accuracy, we interpret model outputs as inputs to budgeting, scheduling, staffing, and contingency planning.

Contributions:

- Build and evaluate ML regression models for effort prediction on a historical dataset [5].
- Report both standard regression metrics and classical effort-estimation metrics used in the effort estimation literature [3], [4].
- Use nested cross-validation to obtain conservative, reproducible performance estimates under tuning.
- Translate estimation performance into project management implications (budget variance, schedule sensitivity, and risk buffers).

II. Background and Related Work

A. Effort Estimation in Project Management

Effort (e.g., person-hours or person-months) is a primary driver of project budgeting and scheduling. Planning decisions such as staffing allocation, milestone feasibility, and contingency reserves depend on effort forecasts. Because effort estimation errors propagate into cost variance and timeline slippage, effort forecasting acts as a decision-support tool rather than a purely technical prediction task.

B. Traditional Approaches

Classical approaches include expert judgment, parametric cost models, and analogy-based estimation. These methods are interpretable and can be applied early, but they may be inconsistent across teams and can degrade when project characteristics differ from calibration data [1], [2]. This motivates data-driven estimation that can adapt to patterns present in historical project datasets.

C. Machine Learning for Software Effort Estimation

ML-based effort estimation has a long research history, including early work exploring learning approaches for effort prediction [1]. Recent studies and systematic reviews report the use of multiple model families: (i) linear and regularized regression, (ii) instance-based learning such as KNN, (iii) kernel methods such as SVR, (iv) tree-based models and ensembles such as random forests, and (v) neural networks [3], [4], [6], [7]. Comparative studies commonly evaluate these models using public datasets and show that relative performance depends on data characteristics, preprocessing, and validation design [2], [8], [9].

D. Datasets, Validation Protocols, and Metrics

A practical challenge is that many public effort datasets are small and heterogeneous, making results dataset-dependent [3], [4]. As a result, k-fold cross-validation (often $k = 10$) is widely used. Alongside standard regression metrics (MAE, RMSE, and R^2), effort-estimation literature frequently reports MMRE and PRED(25) to support comparisons across studies [3], [4]. Some research also emphasizes uncertainty-aware estimation using confidence intervals to support planning under risk [10].

E. Research Gap and Motivation

Many studies focus on predictive performance, but project management requires actionable interpretation for budgeting, staffing, scheduling, and risk reserves. A recurring gap is that prediction results are often reported without a clear translation into planning decisions. This paper addresses that gap by mapping estimation performance into project management implications while maintaining a reproducible ML evaluation workflow.

III. Dataset and Preprocessing

A. Dataset

We use a dataset of 81 software projects with project attributes and an effort target (**Effort**) [5]. Features include experience indicators, project characteristics, function point measures, and a categorical language indicator.

B. Preprocessing

We apply the following preprocessing steps:

- Remove identifier columns (e.g., `id`, `Project`) from modeling.
- One-hot encode `Language` as a categorical feature.
- Standardize numeric features for scale-sensitive models (SVR, KNN).
- Train models on $\log(1+\text{Effort})$ to reduce skew, then invert predictions to the original scale for evaluation.

IV. Methodology

A. Models

We evaluate commonly used regression models in effort estimation research [3], [4]:

- Baseline estimator (mean predictor)
- Linear Regression (LR)
- K-Nearest Neighbors Regressor (KNN)
- Support Vector Regression (SVR)
- Random Forest Regressor (RF)

B. Nested Cross-Validation and Hyperparameter Tuning

To avoid optimistic bias during hyperparameter selection, we use nested cross-validation. The outer loop uses 10-fold cross-validation to estimate generalization performance. Within each outer training fold, an inner 5-fold grid search selects hyperparameters, tuned to minimize MAE (mean absolute error), which is directly interpretable for project planning. Baseline and linear regression models are evaluated without tuning.

Search spaces (summary):

- KNN: $k \in \{3, 5, 7, 9, 11\}$, weights $\in \{\text{uniform}, \text{distance}\}$, $p \in \{1, 2\}$.
- SVR (RBF): $C \in \{1, 10, 100\}$, $\gamma \in \{\text{scale}, 0.1, 0.01\}$, $\epsilon \in \{0.05, 0.1, 0.2\}$.
- RF: trees $\in \{200, 500\}$, depth $\in \{\text{None}, 5, 10\}$, min_leaf $\in \{1, 2, 4\}$, max_features $\in \{\surd, 0.5\}$.

TABLE I
Model performance using nested cross-validation (outer=10, inner=5).

Model	MAE	RMSE	R^2	MMRE	PRED(25)
Baseline	2863.346	4625.066	-0.109	0.832	0.284
LR	2082.499	3707.336	0.287	0.407	0.407
KNN	2123.684	3435.501	0.388	0.534	0.346
SVR	2185.747	3821.916	0.243	0.487	0.358
RF	1921.178	3304.780	0.434	0.459	0.432

C. Evaluation Metrics

We report MAE, RMSE, and R^2, plus effort-estimation metrics MMRE and PRED(25). Let y_i be the true effort and $\hat{y}_i$ the predicted effort:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{1}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{2}$$

$$\text{MRE}_i = \frac{|y_i - \hat{y}_i|}{y_i}, \quad \text{MMRE} = \frac{1}{n}\sum_{i=1}^{n}\text{MRE}_i \tag{3}$$

$$\text{PRED}(25) = \frac{1}{n}\sum_{i=1}^{n}\mathbf{1}(\text{MRE}_i \leq 0.25) \tag{4}$$

V. Results

A. Prediction Performance (Nested CV)

Table I summarizes model performance using nested cross-validation (outer=10 folds, inner=5 folds). Compared to non-nested evaluation, nested CV typically yields more conservative estimates because tuning is confined to training folds.

B. Discussion of Results

The baseline mean predictor performs poorly, yielding a negative R^2 and high MMRE, indicating that naive estimation is not sufficient for reliable planning. Among the evaluated models, Random Forest provides the best overall performance, achieving the lowest MAE (1921.178) and RMSE (3304.780), the highest R^2 (0.434), and the strongest PRED(25) (0.432). This suggests that nonlinear relationships and feature interactions play an important role in predicting effort for this dataset. Linear Regression achieves the best MMRE (0.407), while SVR does not outperform LR or RF under the tuned nested-CV evaluation.

C. Comparison with Prior Work

To support comparison with existing literature, we evaluate models using commonly reported metrics (MAE, RMSE, MMRE, and PRED(25)) and a cross-validation protocol consistent with prior studies [3], [4]. Our findings align with commonly reported observations:

- **No universal best model:** performance depends on dataset characteristics, preprocessing, and validation choices [3], [4], [8].

- **Ensembles are often competitive:** tree-based ensembles such as random forests frequently provide strong performance due to robustness and ability to model feature interactions [2], [4], [6].

Differences between our results and published values may arise from categorical encoding choices, whether the target is modeled in log-space, and the intensity of hyperparameter tuning.

VI. Project Management Implications

A. Budgeting

If r is the labor cost per effort unit (e.g., cost per person-hour or person-month), then:

$$\widehat{\text{Cost}} = r \times \widehat{\text{Effort}} \tag{5}$$

Reducing MAE reduces average budget deviation, improving cost baseline reliability and supporting more accurate cost control.

B. Scheduling and Staffing

A simplified schedule approximation for team capacity c (effort units per time unit) is:

$$\widehat{\text{Duration}} \approx \frac{\widehat{\text{Effort}}}{c} \tag{6}$$

Underestimation implies optimistic schedules and resource shortages; overestimation implies unnecessary slack and potential overstaffing. Project managers can use model error statistics (e.g., MAE/RMSE) to stress-test staffing scenarios and milestone feasibility.

C. Risk and Contingency Reserves

Residual errors can inform contingency buffers. For example, an uncertainty-aware approach can set reserves using a percentile of absolute residuals, reducing the probability of cost/schedule overrun. Confidence-interval-based estimation has also been proposed to support more robust planning under uncertainty [10].

VII. Threats to Validity

- **External validity:** a single dataset may not represent all organizations or modern practices [3], [4].
- **Internal validity:** preprocessing and hyperparameter choices can affect outcomes.
- **Construct validity:** effort measurements may contain noise or inconsistent definitions.
- **Statistical validity:** small sample size (81) increases variance and overfitting risk.

VIII. Conclusion

We presented an ML-based effort estimation study and linked predictive performance to project management decisions. Using a tuned nested cross-validation workflow, Random Forest achieved the strongest overall performance on this dataset, suggesting that nonlinear modeling is beneficial for effort estimation. Future work includes evaluating additional datasets, exploring uncertainty estimation for risk-aware planning, and testing cross-dataset generalization [7], [10].

TABLE II
Most frequent hyperparameter selections across outer folds (nested CV).

Model	Freq.	Hyperparameters (summary)
KNN	3/10	$k = 5$, $p = 1$ (Manhattan), weights=uniform
KNN	3/10	$k = 5$, $p = 1$ (Manhattan), weights=distance
SVR	5/10	RBF, $C = 10$, $\gamma = 0.01$, $\epsilon = 0.2$
SVR	2/10	RBF, $C = 1$, $\gamma = 0.01$, $\epsilon = 0.1$
RF	2/10	trees=200, depth=5, max_feat=0.5, min_leaf=1
RF	2/10	trees=500, depth=None, max_feat=$\sqrt{}$, min_leaf=1

Appendix A
Hyperparameter Stability (Nested CV)

Table II summarizes the most frequent hyperparameter selections across outer folds. This provides a simple indicator of tuning stability; full frequency details can be stored as a supplementary CSV file.

Appendix B
Supplementary Output Files

For reproducibility, the experiment can include the following supplementary artifacts:

- `nested_cv_results.csv`: final metrics per model (MAE, RMSE, R^2, MMRE, PRED(25)).
- `best_params_frequency.csv`: frequency of selected hyperparameters across outer folds.

These files are useful for auditing results, creating plots, and ensuring the reported table values match the executed notebook.

References

[1] K. Srinivasan and D. Fisher, "Machine learning approaches to estimating software development effort," *IEEE Transactions on Software Engineering*, vol. 21, no. 2, pp. 126–137, 1995.

[2] A. BaniMustafa, "Predicting software effort estimation using machine learning techniques," in *2018 8th International Conference on Computer Science and Information Technology (CSIT)*. IEEE, 2018.

[3] A. Ali and C. Gravino, "A systematic literature review of software effort prediction using machine learning methods," *Journal of Software: Evolution and Process*, 2019.

[4] Y. Mahmood, N. Kama, A. Azmi, A. S. Khan, and M. Ali, "Software effort estimation accuracy prediction of machine learning techniques: A systematic performance evaluation," *Software: Practice and Experience*, 2021.

[5] "Software effort estimation dataset (function point / project attributes)," 2026, local CSV used in this study (81 projects, target: Effort). File: fp.csv.

[6] M. Rahman, P. P. Roy, M. Ali, T. Goncalves, and H. Sarwar, "Software effort estimation using machine learning technique," *(IJACSA) International Journal of Advanced Computer Science and Applications*, vol. 14, no. 4, 2023.

[7] T. N. Tran, H. T. Tran, and Q. N. Nguyen, "Leveraging ai for enhanced software effort estimation: A comprehensive study and framework proposal," 2024.

[8] B. Baskeles, B. Turhan, and A. Bener, "Software effort estimation using machine learning methods," IEEE, 2007, iSBN: 1-4244-1364-8.

[9] M. Hammad and A. Alqaddoumi, "Features-level software effort estimation using machine learning algorithms," in *2018 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT)*. IEEE, 2018.

[10] P. L. Braga, A. L. I. Oliveira, and S. R. L. Meira, "Software effort estimation using machine learning techniques with robust confidence intervals," in *Seventh International Conference on Hybrid Intelligent Systems (HIS 2007)*. IEEE, 2007.